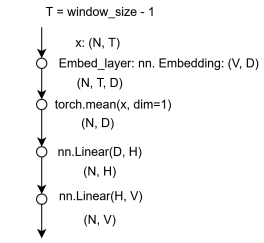
NLP

1. Fundamental concept in NLP: " You shall know a word by the company it keeps." Firth, J.R., 1957.
2. **n-gram**:
    1. **Approximate** the history instead of computing the probability of a word given its entire history.
    2. n-gram can be used as a basis for building probabilistic language models.
       2. when n=1, unigram (bag of words): hello, hyperbole,...
       3. bigram: eat pizza, jump up
       4. trigram: "I like NLP", "I like NLP.".
       5. four-gram, five-gram, six-gram,..., n-gram.
       6. In n-gram LMs, the modeling target is the probability of a sentence. How do you calculate the probability of a sentence? They use n-gram probabilities to calculate the probability of sentences.
    3. unigram or bag-of-words representation for texts.
        1. make a list of all words occuring in the text.
        2. Throw away all words that are too common ("the", "a", "for", "you", ...).
        3. Use stemming.
        4. For each text count how often each word occurs.
    4. n-gram vs n-gram model
        1. n-grams is just a sequence of n tokens.
        2. n-gram model is (n-1)th Markov assumption, i.e. bigram model is 1st order Markov assumption.
3. **word2vec**
    1. word2vec and similar word embeddings are a good example of **self-supervised learning** where the training labels are determined by the input data.
        1. word2vec models predict a word from its surrounding words (and vice versa). Unlike traditional supervised learning, the class labels are not separate from the input data.
        2. we're training a simple neural network with a single hidden layer to perform a certain task, but then we're not actually going to use that neural network for the task we trained it on! Instead, the goal is actually just to learn the weights of the hidden layer -- these weights are actually the "word vectors" that we're trying to learn.
    2. **Autoencoders** are another example of self-supervised learning, as they are trained to shrink and reconstruct their inputs.
        1. when we train an autoencoder to compress an input vector in the hidden layer, and decompress it back to the original in the output layer, we strip off the output layer (the decompression step) and just use the hidden layer -- it's a trick for learning good image features w/out having labeled training data.
    3. why not just use SVD? because SVD is much more computationally expensive compared to word2vec.
        1. SVD: Computational cost scales quadratically for n x m matrix:
            1. O(mn^2) flops when n < m.
            2. Bad for millions of words or documents.
            3. Hard to incorporate new words or documents.
            4. Different learning regime than other DL models.
    4. We will train a classifier on a binary prediction task: is the word w likely to show up near parrot?
    5. Skip-gram:
        1. Predict surrounding words if you give me one word.
        2. Gives good representation for rare words.
        3. Use the target word & a neighboring context word (from a corpus) as positive examples.
        4. Randomly sample other words as negative examples.
        5. Train a classifier to distinguish those two cases.
        6. Use the learned weights as the embeddings.
        7. minimize negative log likelihood, $L_{CE} = -\log \left[P(+|w, c_{pos}) \prod_{i=1}^{k} P(-|w, c_{neg}) \right]$
        8. Negative sampling converts the problem:
            1. From: A massive multi-class classification task (which word is the right context word out of all words in the vocabulary)
            2. To: A small set of binary classification tasks (is this pair a true context pair or a random one?)
            3. Instead of updating the weights for |V| words, the model updates the weights for:

    1. 1 positive word
    2. A small, fixed number of k negative samples (typically k=5 or 20)
  4. This reduces the computational complexity from O(|V|) to O(k), making it feasible to train word embeddings on massive datasets.
6. CBOW
  1. Predict one word if you given the surrounding words.
  2. Faster to train, good on large text corpus.
  3. Better representation for frequent words.

word2vec: CBOW model



criterion: nn.CrossEntropyLoss()

```
sentences = ['cat chases mouse',
             'cat catches mouse',
             'cat eats mouse',
             'mouse runs into hole',
             'cat says bad words',
             'cat and mouse are pals',
             'cat and mouse are chums',
             'mouse stores food in hole',
             'cat stores food in house',
             'mouse sleeps in hole',
             'cat sleeps in house',
             'cat and mouse are buddies',
             'mouse lives in hole',
             'cat lives in house']
stop_words = set(stopwords.words("english"))
```

```
V = vocab_size = 19
D = embed_dim = 2
H = hidden_dim = 10

word_embeddings.shape: (19, 2)
      len(word2idx.keys()): 19
```

```
Training:
batch_size = 4, window_size = 3

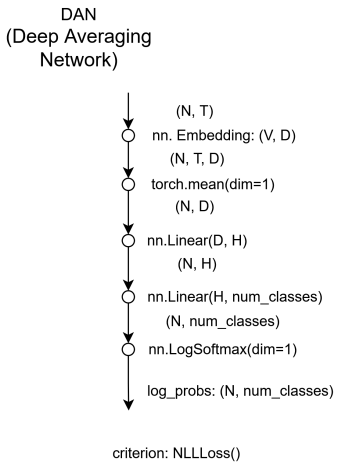
x.shape, y torch.Size([4, 2]) torch.Size([4])
x, y tensor([[0, 5],
      [1, 3],
      [5, 0],
      [0, 6]]) tensor([1, 5, 3, 3])
indices: x_b: [0 5], y_b: 1
words: x_b: ['UNK', 'eats'], y_b: cat
indices: x_b: [1 3], y_b: 5
words: x_b: ['cat', 'mouse'], y_b: eats
indices: x_b: [5 0], y_b: 3
words: x_b: ['eats', 'UNK'], y_b: mouse
indices: x_b: [0 6], y_b: 3
words: x_b: ['UNK', 'runs'], y_b: mouse

x.shape, y torch.Size([4, 2]) torch.Size([4])
x, y tensor([[1, 3],
      [5, 0],
      [0, 6],
      [3, 7]]) tensor([5, 3, 3, 6])
indices: x_b: [1 3], y_b: 5
words: x_b: ['cat', 'mouse'], y_b: eats
indices: x_b: [5 0], y_b: 3
words: x_b: ['eats', 'UNK'], y_b: mouse
indices: x_b: [0 6], y_b: 3
words: x_b: ['UNK', 'runs'], y_b: mouse
indices: x_b: [3 7], y_b: 6
words: x_b: ['mouse', 'hole'], y_b: runs
```

```
x.shape, y torch.Size([4, 2]) torch.Size([4])
x, y tensor([[5, 0],
      [0, 6],
      [3, 7],
      [6, 0]]) tensor([3, 3, 6, 7])
indices: x_b: [5 0], y_b: 3
words: x_b: ['eats', 'UNK'], y_b: mouse
indices: x_b: [0 6], y_b: 3
words: x_b: ['UNK', 'runs'], y_b: mouse
indices: x_b: [3 7], y_b: 6
words: x_b: ['mouse', 'hole'], y_b: runs
indices: x_b: [6 0], y_b: 7
words: x_b: ['runs', 'UNK'], y_b: hole

x.shape, y torch.Size([4, 2]) torch.Size([4])
x, y tensor([[0, 6],
      [3, 7],
      [6, 0],
      [0, 8]]) tensor([3, 6, 7, 1])
indices: x_b: [0 6], y_b: 3
words: x_b: ['UNK', 'runs'], y_b: mouse
indices: x_b: [3 7], y_b: 6
words: x_b: ['mouse', 'hole'], y_b: runs
indices: x_b: [6 0], y_b: 7
words: x_b: ['runs', 'UNK'], y_b: hole
indices: x_b: [0 8], y_b: 1
words: x_b: ['UNK', 'says'], y_b: cat
```

    4.
7. Applying embeddings
  1. Word embedding algorithm works uses the idea based on **contexts**. For example "I painted the bench ___". The blank can be filled with colour: red, green, etc. or it could also be today, but today is not a colour. The main takeaway is that context is closely related to meaning. Another way that context can help us is that if two words happen to always appear in the same context at once.
  2. Approach 1: learn these embeddings as parameters from your data
    1. often works pretty well.
    2. Using dense feature representations is better than using sparse feature representations. For example using a feature embedding of length 50-300 is better than using 3000, since the neural network has far less parameters to learn.
  3. Approach 2: initialize word embeddings using GloVe, keep fixed
    1. faster because no need to update these parameters.
    2. Problem with antonyms.
  4. Approach3: Initialize word embeddings GloVe, fine-tune
    1. works best for some tasks
    2. standard in sentiment analysis
  5. Can also evaluate embeddings intrinsically on tasks like word similarity.
    1. Deep averaging Network




```
Training:
data.shape: (batch_size, T), target.shape: (batch_size,)
...
data.shape: (batch_size, 21), target.shape: (batch_size,)
data.shape: (batch_size, 43), target.shape: (batch_size,)
data.shape: (batch_size, 28), target.shape: (batch_size,)

  SentimentExample:
  attributes:
  - words: type: List[string]
  - label: type: int

  train_exs: training examples: type: List[SentimentExample]:
  [['fantastic', '!']; label=1,
  ['a', 'thought-provoking', 'picture','.']; label=1,
  ['a', 'stylish', 'thriller','.']; label=1,
  ['almost', 'peerlessly', 'unsettling','.']; label=0,
  ['delirious', 'fun','.']; label=1,
  ['exciting', 'and', 'well-paced','.']; label=1, ...]

  Build indexer from the whole corpus (train_exs):

  indexer = Indexer()
  indexer.add_and_get_index("UNK")
  for ex in train_exs:
      for word in ex.words:
          indexer.add_and_get_index(word)
```

```
data.shape, target.shape torch.Size([4, 19]) torch.Size([4])
data, target tensor([[ 1, 1727, 3573, 3574, 3575, 3, 197, 71, 176, 33, 0, 0, 0, 0,
0, 0, 0, 0, 0], [4722, 290, 3105, 75, 1, 4723, 1194, 33, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0], [ 899, 26, 159, 563, 26, 304, 608, 2308, 33, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [ 118,
441, 242, 19, 4393, 4473, 5, 4474, 267, 19, 1323, 1037, 37, 3360, 279, 9, 729,
546, 33]]) tensor([1, 1, 0, 0])

data.shape, target.shape torch.Size([4, 28]) torch.Size([4])
data, target tensor([[ 86, 495, 5, 11442, 1, 8773, 104, 845, 1, 443, 104, 1,
5571, 474, 6, 4372, 33, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [ 487, 3422, 3423, 3, 248,
67, 23, 1794, 3495, 1523, 5, 19, 10, 26, 4023, 1864, 33, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0], [ 11, 605, 13, 3, 19, 7641, 9876, 2907, 4047, 93, 1, 823, 5, 1678, 278,
37, 1, 234, 33, 0, 0, 0, 0, 0, 0, 0, 0, 0], [ 1, 228, 9, 1122, 1273, 210, 187, 104,
1, 337, 26, 334, 8857, 9, 8858, 994, 5, 8859, 9, 6073, 6200, 8860, 14, 8861, 9,
8862, 3342, 33]]) tensor([0, 0, 0, 1])

data.shape, target.shape torch.Size([4, 43]) torch.Size([4])
data, target tensor([[ 19, 1046, 13860, 71, 501, 10432, 9585, 12607, 1520, 33,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0], [ 304, 1229, 67, 33, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [ 268, 14, 902, 37, 1, 2701,
2702, 210, 41, 2246, 26, 14, 10535, 9, 419, 3, 41, 2854, 26, 118, 9, 19, 809,
1592, 15, 41, 240, 37, 1, 228, 63, 1985, 26, 342, 104, 1, 4088, 63, 3, 19,
12093, 6407, 33], [ 19, 4724, 26, 3592, 26, 4640, 1975, 33, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]])
tensor([0, 0, 0, 1])

data.shape, target.shape torch.Size([4, 21]) torch.Size([4])
data, target tensor([[14307, 6372, 5279, 26, 1, 228, 3, 41, 10185, 1058, 14,
11053, 1804, 118, 1123, 3146, 86, 4751, 104, 14308, 33], [ 19, 228, 86, 3405,
26, 273, 23, 384, 86, 6637, 232, 33, 0, 0, 0, 0, 0, 0, 0, 0, 0], [ 1800, 118, 425,
5615, 33, 303, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [ 5278, 359, 5279, 26,
328, 9, 240, 5, 3494, 1, 249, 33, 0, 0, 0, 0, 0, 0, 0, 0, 0]]) tensor([0, 0, 1, 1])
```

2.

4. **Vocabulary**
    1. Think about it as a dictionary where each key is an individual token that appear in our corpus with an index. Tokens are not clearly defined but tokens are as we are converging to is a subword. And subword is not really linguistically explainable, it can be either for example here 'low' is a full word, 'newer' is a full word, but 'ne' is not a full word. Taking for example from subwords produced by BPE, learned encoding:
    2. Corpus is a collection of texts. Usually, to simplify when trying algorithms we can treat each text as a just sentence. So a corpus is just a list of sentences: type: List[sentences: type: string]
    3. Corpus:
        1. 5: l o w _
        2. 2: l w w e s t _
        3. 6: ne w er_
        4. 3: w i d er_
        5. 2: ne w _
    4. Note: the number in the beginning of each line is the number of occurrences of each of these words in the corpus (but they are not needed, can be ignored).
    5. Vocabulary:
        1. _, d, e, i, l, n, o, r, s, t, w, er, er_, ne
    6. BPE:
        1. merge: (ne, w), current*vocabulary: _, d, e, i, l, n, o, r, s, t, w, er, er*, ne, new
        2. merge: (l, o), current*vocabulary: _, d, e, i, l, n, o, r, s, t, w, er, er*, ne, new, lo
        3. merge: (lo, w), current*vocabulary: _, d, e, i, l, n, o, r, s, t, w, er, er*, ne, new, lo, low
        4. etc.
    7. OOV
        1. OOV: words that were seen very rarely during training or not even at all.
        2. Closed vocabulary models: unable to produce word forms unseen in training data. As opposed to this we have Open vocabulary models.
        3. UNK tokens
            1. Historically UNK was used to replace any tokens that is not part of the training vocabulary.
            2. But we shouldn't generate UNK when generating text (imagine ChatGPT doing this).
            3. UNKs don't give features for novel words that are useful anchors of meaning.
        4. Tokenization:
            1. whitespace tokenization: simple, but does not perform well. Also some language do not have spaces such as Chinese, Korean, etc.
        5. H2O/water have the same meaning, but no one will say "take your surf board and jump into H2O" because it will be weird.
        6. Distributional Hypothesis
5. Vector search vs Semantic search
    1. How vector search works
        1. Vector search operates by converting data items into vectors within multi-dimensional vector space. This transformation is achieved using embedding models such as word2vec, Glove, or transformers, which maps words

or sentences to dense numerical vectors. Similarly, images can be embedded using CNNs.
    2. Once the data is represented as vectors, the search process involves computing distances between these vectors to measure similarity. Common distance metrics include:
        1. Euclidean distance: Using Pythagorean theorem to measure straight line between two points.
        2. Cosine similarity: Calculates the cosine angle between two vectors, focusing only on their direction rather than magnitude.
        3. Dot product: Evaluates the correlation or alignment between vectors.
    3. When a query is submitted, it is converted into a vector using the same embedding model.
    4. The system then identifies the closest vectors in the dataset, using approximate nearest neighbor (ANN) algorithms to expedite searches within large datasets.
    5. Modern vector search libraries: FAISS, Qdrant.
2. How semantic search works
    1. Semantic search begins with preprocessing both the query and the database content. This involves parsing and embedding textual data using NLP models.
    2. Common techniques include tokenization, stemming, and use of transformer-based models like BERT or GPT-2 to generate context-aware embeddings.
    3. These embeddings capture the semantic relationships between words and phrases, accounting for polysemy (words with multiple meanings) and synonyms. For example, “bank,” as a financial institution, and “bank,” as a river’s edge, are disambiguated through contextual understanding.
    4. The search engine compares the query embedding with embeddings of database items, calculating similarity using techniques such as cosine similarity or dot product. Results are ranked based on relevance scores derived from these comparisons.
6. Static vs contextualized embeddings
    1. Static means we have one fixed embedding for each word in the vocabulary.
        1. Examples:
            1. word2vec: only using words, not subwords, can stuck with the problem of unknown words and so on.
            2. Fasttext [Bojanowski et al., 2017]: can take into account subwords information, basically an extension of skip-gram so that it works with subwords rather than individual words.
            3. Glove [Pennington et al., 2014]
        2. Can be used if we don't have a very complex task that requires a lot of reasoning, then using static embeddings and a simple model like logistic regression can be a very good choice for our classification task.
    2. Contextualized word embeddings in pretrained Language Models and BERT: for example the word "basin" can appear in different contexts based on the sentences. So each one of them will have a different embedding based on the context of each sentence.
7. Visualizations:
    1. Project embeddings to a 2D space and visualize them.
    2. t-SNE.
8. TF-IDF
    1. Term Frequency (TF) measures the frequency of a term within a document. It is calculated as the ratio of the number of times a term occurs in a document to the total number of terms in that document. The goal is to emphasize words that are frequent within a document.

$$TF(t,d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

    2.
    3. Inverse Document Frequency (IDF) measures the rarity of a term across a collection of documents. It is calculated as the logarithm of the ratio of the total number of documents to the number of documents containing the term. The goal is to penalize words that are common across all documents.

$$IDF(t,D) = \log\left(\frac{\text{Total number of documents in the corpus } N}{\text{Number of documents containing term } t}\right)$$

    4.
    5. TF-IDF score for a term in a document is obtained by multiplying its TF and IDF scores:
    $TF-IDF(t,d,D) = TF(t,d) \times IDF(t,D)$.
9. Point-wise mutual information (PMI)
    1. It turns out that frequency is not the best measure of association between words.
    2. One problem is raw frequency is very skewed and not very disciminative.

3. We'd like context words that are particularly informative about the target word.
4. The best weighing or measure of association between words must tell us how much more often than chance the two words co-occur -- Pointwise mutual information is such a measure.
5. It is based on the notion of mutual information.

10. stemming vs lemmatization
    1. Spacy uses lemmatization instead of stemming.
    2. Porter stemming `from nltk.stem import PorterStemmer`.
11. Preprocessing steps:

    1. use regular expression, re.sub to remove old style retweet, hyperlink, hashtags, etc.
    2. tokenize
    3. lowercasing
    4. remove stop words
    5. remove punctuations,

    ```
    import string
    string.punctuation
    ```

    6. stemming or lemmatization

12. countvectorizer vs tfidfvectorizer.
13. **Language Model**
    1. a model is something that simplifies the world to make the computation feasible, an abstract and simplified version of a subject, make calculation feasible.
    2. Probabilistic model: calculate the probability of a random event.
    3. Take coin tossing for example:
        1. abstract away factors: models: P(head), P(tail)
        2. the modeling target is the probability of the values, the probability of head and probability of tail
        3. After we have the modeling target what is the next step?
        4. The next step is to describe the function to calculate the scores. This process is called parametrization.
    4. MLE
        1. Coin tossing - counting relative frequencies:
        2. what we do intuitively
        3. head: k times, tail: N-k times, P(head)=k/N, P(tail)=(N-k)/N
        4. represent the probability by counting relative frequency -> this is the intuition.
        5. what we've actually done often w/out thinking:
            1. what we model formally:
                1. parametrization: $P(head) = \theta$, $P(tail) = 1 - \theta$.
                2. training: D : $\{y_1, y_2, \ldots, y_N\}$, #head=k, #tail=N-k, $\theta = \frac{k}{N}$.
            2. In our mind we have one model parameter called θ, and the way we calculate the probability of coin facing up is $P(head) = \theta$, then how do we calculate $P(tail) = 1 - \theta$.
        6. Model parametrization
            1. we define some model parameters to specify a scoring function to calculate your target probabilities. Then how do we train the model? the data is our experiments, we throw the coin N times we get k times head. By training the model we mean we decide the value of model parameters. How do we decide the value of model parameters? k times out of N, so that's the value of my $\theta \rightarrow$ this is the training process.
            2. 3 steps:
                1. decide modeling target.
                2. parametrization, decide scoring function and how to calculate it
                3. get a set of data and specify the value of data, this is called model training.
            3. The theory: principle of maximum likelihood
                1. Training data: D : $\{y_1, y_2, \ldots, y_N\}$
                2. training example (instance): $y_i \in \{hand, tail\}$
                3. parameter: $P(head) = \theta$
                4. condition: the tosses are independent and identical (iid) distributed. Note: iid means n experiments are conditionally independent to each other. Only then it will be a binomial distribution.

5. training objective: the log likelihood: $\hat{\theta} = \arg\max P(D) = \arg\max \log(P(D))$
6. Training is to find your model parameter value from your data, and maximum likelihood estimation is to maximize likelihood of your data to find the parameter.
7. Derivation:
    1. The likelihood is $P(D) = \theta^k(1-\theta)^{N-k}$
    2. To maximize it let $\frac{\partial P(D)}{\partial \theta} = 0$, we have:
        1. $\frac{\partial P(D)}{\partial \theta} = \frac{\partial \log(\theta^k(1-\theta)^{N-k})}{\partial \theta}$
        2. $= \frac{\partial(k\log\theta + (N-k)\log(1-\theta))}{\partial \theta}$
        3. $= \frac{k}{\theta} - \frac{N-k}{1-\theta} = 0$
        4. $\theta = \frac{k}{N}$
        5. maximizing $P(D)$ without the logarithm is more challenging, but using the log is easier
        6. So, by saying we'll just toss a coin 100 times, and if 53 out of 100 are heads, we'll say $p = 0.53$, but actually we've done three things in the background.
5. Language model can be used to analyze or generate natural language sentences.
6. A probabilistic LM measures the probability of natural language sentences, by means of simpler patterns, such as:
    1. words: "thanks" is more probable than "Markov"
    2. phrases: P("eat pizza")>P("drink pizza")
    3. sentences: P("he said hi")>P("said he hi")
7. Unigram LMs
    1. model the sentence as a set of unigrams or model the sentence as a bag of words.
    2. assume that every word in the sentence is iid.
    3. Its like we randomly generate a word then randomly generating a second word, and then third word to form a sentence.
    4. This is a set of categorical experiments, it's a multinomial distribution.
    5. Or if you care about the order, it's just a multinomial distribution without the coefficient.
    6. Modeling target: P(s)
    7. Parametrization - iid assumption between words in a sentence: $P(s) = P(w_1)P(w_2)P(w_3)\dots P(w_n) = \prod_{i=1}^{n} P(w_i)$
    8. Parameter type: The probability of a word
    9. Parameter instances: $|V|$, how many independent parameters are in my model? the number of parameters is the vocabulary size.
    10. Training: just as we trained P(w) word drawing.
    11. because every model parameters is a separate word probability, we can train the model by using MLE (maximum likelihood estimation).
    12. add-one smoothing (more principal way to assign probability to [OOV])
        1. introduce a special <OOV> token to represent OOV words.
        2. recalculate the counts of all words in D.
        3. $P(w) = \frac{(\#w \in D)+1}{\sum_{w' \in V}((\#w' \in D)+1)} = \frac{(\#w \in D)+1}{|V| + \sum_{w' \in V}(\#w' \in D)}$
    13. add-α smoothing
        1. α is the model hyperparameter.
        2. Hyperparameter:
            1. fixed in advance and not trained during training e.g. learning rate.
            2. can be tuned, selected empirically to improve performance.
14. Perplexity
    1. Accuracy doesn't make sense, instead, evaluate Language Modeling (LM) on the likelihood of held-out data.
    2. Intuitively, **perplexity can be understood as a measure of uncertainty**. Consider a language model with an entropy of three bits, in which each bit encodes two possible outcomes of equal probability. This means that when predicting a symbol, that LM has to choose among $2^3 = 8$ possible options.
    3. Perplexity is a measure for evaluating language models, not a measure to evaluate sentiment classifiers, for example .
    4. Actually, is the exponential of the average negative log likelihood (lower is better)
    5. $\text{perplexity}(w_1, w_2, \dots, w_n) = P(w_1 w_2, \dots, w_n)^{-\frac{1}{n}} = \left(\prod_{i=1}^{n} P(w_i | w_1 \dots w_{i-1})\right)^{-\frac{1}{n}}$
    6. Usually only reported in LM research papers, not elsewhere.
    7. Perplexity usually ranges from 10-200.

8. Even with low perplexity it doesn't mean that later on your language model will be great for downstream tasks, people have shown that the perplexity and accuracy for downstream tasks does not necessarily correlate.

15. **Text Classification**
    1. Producing a feature vector $x \rightarrow f(x)$, i.e. using nn.Embedding. Then, perceptron and logistic regression $m(f(x)) = \hat{y} \in \{+1, -1\}$. Where m is the model (perceptron, logistic regression, or SVM).
    2. **Logistic Regression**
        1. First method
            1. Corpus:
                1. I am happy because I am learning NLP, +
                2. I am happy, +
                3. I am sad, I am not learning NLP, -
                4. I am sad, -
            2. Feature extraction
                1. Test sentence: "I am sad, I am not learning NLP"
                2. $x = \begin{bmatrix} 1 \\ 8 \\ 11 \end{bmatrix}$. $x[1] = 8$ is the sum of pos. freqs. $x[2]$ is the sum of neg. freqs (see the table of word frequencies).
                3. Table of word frequencies

Vocabulary	frequency	
	+	-
I	3	3
am	3	3
happy	2	0
because	1	0
learning	1	1
NLP	1	1
sad	0	2
not	0	1

                4. Test sentence: "I am happy because I am learning NLP"
                    1. Preprocessing:
                        1. removing stop words, punctuations, handles, URLs.
                        2. stemming.
                        3. lower casing.
                    2. After preprocessing: [happy, learn, NLP]
                    3. ```
                       ```
        2. Second method
            1.
    3. **Naive Bayes**
        1. $freqs(w, 1)$: frequency of a specific word where the class label is 1 (in the $freqs$ table).
        2. $freqs(w, 0)$: frequency of a specific word where the class label is 0 (in the $freqs$ table).
        3. N_{pos}: frequency of all the words (in the $freqs$ table) with the class label 1.
        4. N_{neg}: frequency of all the words (in the $freqs$ table) with the class label 0.
        5. $|V|$: number of unique words in the corpus (number of keys in the $freqs$ table).
        6. D_{pos}: number of sentences with class label 1 (in the corpus).
        7. D_{neg}: number of sentences with class label 0 (in the corpus).
        8. D: corpus: x_{train}: List[sentences: type: string], y_{train}: List[label: type: int]
        9. $freqs$ table: keys: tuple: (word: type: string, label: type: int), values: frequency (count): type: int.
        10. Example [5]:

| | Doc | Words | Class | | --- | --- | ----- | --- | | Training | 1 | Chinese Beijing Chinese | c | | | 2 | Chinese Chinese Shanghai | c | | | 3 | Chinese Macao | c | | | 4 | Tokyo Japan Chinese | j | | Test | 5 | Chinese Chinese Chinese Tokyo Japan | ? |
                       ```

11. Build freq table
    `| word | class 0 | class 1 | | ------ | ----- | ----- | |Chinese | 5 | 1 | |Beijing | 1 | 0 | |Shanghai | 1 | 0 | |Macao | 1 | 0 | |Tokyo | 0 | 1 | |Japan | 0 | 1 |`
12. $P(w|c) = \frac{count(w,c)+1}{count(c)+|V|}$. From $p(w|c) = \frac{(\#(w,c)\in D)+1}{\sum_{w'\in V}((\#(w',c)\in D)+1)}$
13.

16. Top-k sampling vs Top-p sampling
    1. Top-k sampling
        1. Fixed number of candidates.
        2. Better for more deterministic outputs.

Excercises

Problem 1: TF-IDF Calculation

Assignment:
Consider the following corpus of three documents:

- **D1:** "the ship sailed on the sea"
- **D2:** "the sea is vast and deep"
- **D3:** "the ship was deep in the sea"
  Calculate the TF-IDF score for the term "sea" in Document D1. Use the natural logarithm for the IDF calculation.

Solution:
The formula for TF-IDF is:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

Where the Inverse Document Frequency (IDF) is calculated as:

$$\text{IDF}(t) = \log\left(\frac{N}{|\{d \in D : t \in d\}|}\right)$$

Step 1: Calculate Term Frequency (TF)
The **term** "sea" appears 1 time in Document D1, which has a total of 6 words.

$$\text{TF}(\text{”sea”}, D1) = \frac{1}{6}$$

Step 2: Calculate Inverse Document Frequency (IDF)

- Total number of documents in the corpus (N) = 3.
- Number of documents containing the term "sea" = 3 (D1, D2, D3).

$$\text{IDF}(\text{”sea”}) = \log\left(\frac{3}{3}\right) = \log(1) = 0$$

Step 3: Calculate the Final TF-IDF Score

$$\text{TF-IDF}(\text{”sea”}, D1) = \text{TF}(\text{”sea”}, D1) \times \text{IDF}(\text{”sea”}) = \frac{1}{6} \times 0 = 0$$

The TF-IDF score is 0. This indicates that the term "sea" is not a good differentiator for this corpus as it appears in every document.

Problem 2: The Chain Rule for Language Modeling

Assignment:
The probability of a sequence of words $w_1, w_2, \ldots, w_n$ is given by the chain rule of probability:

$$P(w_1, \ldots, w_n) = \prod_{i=1}^{n} P(w_i|w_1, \ldots, w_{i-1})$$

Explain the practical significance of this formula in the context of an NLP task such as machine translation or speech recognition. Why is this decomposition important?

Solution:
The chain rule is foundational to statistical language modeling. Its significance lies in decomposing the problem of calculating the joint probability of an entire sequence of words (which is computationally intractable to estimate directly) into a product of conditional probabilities.

In practice, a language model uses this rule to predict the next word given the previous words. For instance:

- **Machine Translation:** When generating a translated sentence, the model generates one word at a time. At each step, it calculates the probability of all possible next words given the sequence already generated. The chain rule provides the mathematical framework to score the likelihood of the entire generated sentence, helping the system choose the most probable (and thus, most fluent) translation.
- **Speech Recognition:** An audio signal can correspond to multiple possible word sequences (e.g., "recognize speech" vs. "wreck a nice beach"). The system uses a language model to calculate the prior probability of each candidate word sequence. The sequence with the higher probability according to the chain rule is considered more likely to be the correct transcription.

This decomposition is crucial because estimating $P(w_i|w_1, \ldots, w_{i-1})$ is far more feasible than estimating $P(w_1, \ldots, w_n)$ for the entire vocabulary and all possible sentence lengths.

Problem 3: Vector Space Models and Semantic Similarity

Assignment:
In the Vector Space Model (VSM), documents are represented as vectors. Explain what the cosine similarity between two document vectors, d_1 and d_2, represents. If the cosine similarity between the vectors for "Artificial Intelligence" and "Machine Learning" is high (e.g., > 0.8), while the similarity between "Artificial Intelligence" and "Astrophysics" is low (e.g., < 0.2), what does this imply?

Solution:
The cosine similarity between two vectors measures the cosine of the angle between them. In the context of a VSM, it represents the **semantic similarity** of the documents. It is not concerned with the magnitude (e.g., length of the document) but rather the orientation of the vectors.

- A cosine similarity score close to 1 implies that the angle between the vectors is very small, meaning the documents are pointing in a similar direction in the vector space. This indicates a high degree of semantic or topical similarity.
- A cosine similarity score close to 0 implies that the vectors are nearly orthogonal, indicating they have very little topical overlap.
- A score close to -1 would imply they are opposites, which is rare in document analysis using non-negative weighting schemes like TF-IDF.

Implication of the Example:
A high cosine similarity (> 0.8) between "Artificial Intelligence" and "Machine Learning" implies that these two documents share a significant amount of related terminology and are considered topically very similar by the model. Conversely, the low similarity (< 0.2) between "Artificial Intelligence" and "Astrophysics" implies they share very few common terms and are topically distinct. This demonstrates the VSM's ability to capture thematic relationships within a corpus.

References:

1. CS 6340, NLP, UofU, Ana Marasović.
2. Hongseo Jang, https://github.com/UTAH-CS-ARCHIVE/cs6340-nlp/tree/main.
3. https://readmedium.com/tf-idf-in-nlp-term-frequency-inverse-document-frequency-e05b65932f1d.
4. Vector search vs Semantic search. https://www.instaclustr.com/education/vector-database/vector-search-vs-semantic-search-4-key-differences-and-how-to-choose/.
5. Multinomial Naive Bayes - A worked example, Dan Jurafsky, youtube, uploaded by Rafael Merino Garcia.
6. Han Cheol Moon, Notes on NLP, https://han8931.github.io
7. word2vec, https://machinelearning.wtf/terms/self-supervised-learning/.
8. word2vec, https://mccormickml.com/2016/04/19/word2vec-tutorial-the-skip-gram-model/
9. Vector Semantics and Embeddings, CUHK.
10. Coursera, NLP Specialization.
11. https://jiyuanliu.netlify.app/teaching/word_embed/

12.